# AuthDrift — 3-Objective Prototype Implementation Plan

Detailed architecture, team allocation, Git workflow, development sequence, interfaces, testing, and final demonstration plan.

## 1. Project Context

AuthDrift is a lightweight, vendor-neutral monitoring layer for analysing authentication assurance across a user's authentication journey. The project does **not** replace an identity provider, SIEM, or authentication policy system.

Its core idea is to:

- Correlate authentication events
- Model the relative assurance of authentication methods
- Reconstruct the authentication journey
- Identify meaningful reductions in assurance

For the first prototype, the guide requires the system to demonstrate at least **three objectives**:

1. Collect and normalize authentication events
2. Model authentication methods using relative assurance levels
3. Reconstruct complete authentication journeys for individual users

A minimal drift-detection layer should also be included because it demonstrates the central purpose of AuthDrift.

## 2. Prototype Scope

| Included | Not required initially |
|---|---|
| Keycloak controlled authentication environment | Real Microsoft/Google/Okta production integration |
| Authentication event collection | Enterprise-scale real-time streaming |

| | |
|---|---|
| Common normalized event schema | Large multi-provider ingestion platform |
| Relative assurance assignment | Complex ML-based assurance model |
| User journey reconstruction | Full production SIEM replacement |
| Basic assurance-drift detection | Automatic account blocking |
| Controlled test scenarios | Production security response automation |

## 3. Complete Prototype Pipeline

**Prototype sequence:**

```
KEYCLOAK → Authentication Events → Event Normalization → Assurance Assignment
→ Authentication Journey Reconstruction → Basic Drift Detection → Testing
```

**Broader synopsis pipeline:**

```
Authentication Logs → Event Normalization → Authentication History/Timeline
→ Assurance Assignment → State/Sequence Analysis → Drift Detection
→ Risk Scoring → Explainable Alert → Dashboard
```

The three-objective prototype implements the front portion and a minimal drift demonstration; the remaining stages can be added after the core works.

## 4. Objective 1 — Collect and Normalize Authentication Events

**Goal:** Obtain authentication events from Keycloak and convert them into a common structure that the rest of the system can process.

**Owner: Tashi**

- Set up the Keycloak controlled test environment
- Create test users and the required authentication scenarios
- Generate authentication events for success/failure and relevant contextual information where available
- Extract/export the raw events

- Implement the initial raw-event-to-common-schema conversion

**Common event schema:**

```
event_id, user_id, timestamp, event_type, authentication_method,
outcome, device_id, location, authenticator_id
```

> The exact fields should be frozen before coding. If a field is unavailable in a particular source, it should be represented consistently rather than silently invented.

## 5. Objective 2 — Assurance Assignment

**Goal:** Represent the relative strength/assurance of each authentication method so the system can detect a reduction across a journey.

**Owner: Balaa**

Define the prototype assurance model and implement the mapping from authentication method to relative assurance. The synopsis requires relative assurance levels but does not prescribe the final numerical scale — the final values must be justified during implementation.

| Method | Assurance |
|---|---|
| Passkey | HIGH |
| OTP | MEDIUM |
| Password | LOW |

*Illustrative categories, not a finalized/mandated scale.*

## 6. Objective 3 — Authentication Journey Reconstruction

**Goal:** Stop treating authentication events as isolated records; reconstruct a user's sequence of authentication activity.

**Owners: Rohan + Balaa**

- Group normalized events by user and defined journey/session window
- Order events chronologically using timestamps
- Represent the resulting sequence as an authentication timeline
- Expose the journey in a structure that the detection engine and UI can consume

**Example:**

```
Passkey → FAILED → Password → SUCCESS → New Device
```

This becomes **one authentication journey**, not three unrelated log entries.

## 7. Minimal Drift Detection Layer

Although the guide requires only three objectives, the prototype should include a minimal drift detector — otherwise the demo proves data processing but doesn't visibly demonstrate the central AuthDrift idea.

The initial detector can be **rule-based**: compare assurance levels between relevant authentication events and identify a meaningful decrease.

```
HIGH assurance passkey → LOWER assurance password → assurance decrease → drift detec
```

> Do not claim every fallback is an attack. Legitimate recovery, device migration, and authenticator replacement can cause assurance changes. The prototype should identify and explain the observed transition; more sophisticated contextual risk decisions belong to later stages.

## 8. Team Branch Allocation

| Person / Branch | Primary Ownership | Concrete Deliverables |
|---|---|---|
| **Balaa** — `balaa-assurance-detection` | Assurance + detection | Assurance model, assurance assignment, basic drift detection, output contract |

| | | |
|---|---|---|
| **Tashi** — `tashi-keycloak-events` | Authentication + event pipeline | Keycloak setup, scenarios, raw event extraction, normalization |
| **Rohan** — `rohan-journey-ui` | Journey + interface | Journey reconstruction, timeline representation, basic UI, display of detection results |
| **Varun** — `varun-testing` | Testing + validation | Test cases, test data, expected/actual comparison, regression tests, validation |

## 9. How the Branches Connect

- **Tashi:** Keycloak → raw events → normalized events
- **Balaa:** normalized events → assurance assignment → drift result
- **Rohan:** normalized events → journey/timeline → display of results
- **Varun:** controlled scenarios → expected result → execute prototype → actual result → pass/fail

The branches must not become isolated mini-projects. The common event schema and output contracts are shared interfaces. The final `main` branch contains the integrated system.

## 10. GitHub Workflow — Setup & Rules

- Create one repository for AuthDrift
- Protect `main`; no direct experimental pushes to `main`
- Create one working branch per person
- Commit small, meaningful changes with clear messages
- Push the branch and open a pull request when a module is ready
- Review interface compatibility before merging
- Merge only working, tested code into `main`
- After integration, run the complete prototype test suite

**Recommended branch names:**

```
balaa/assurance-detection
tashi/keycloak-events
```

```
rohan/journey-ui
varun/testing
```

## 11. Local Branch Creation + Developing in Your Tool

Everyone starts the same way, then works inside whichever tool they use — VS Code, Claude Code, Codex CLI, or Antigravity. The Git commands underneath are identical regardless of tool; the tool just drives the terminal/editing loop.

### A. One-time repo setup (owner)

```
git init AuthDrift
cd AuthDrift
git add .
git commit -m "Initial commit: project scaffold"
git branch -M main
git remote add origin https://github.com/<org-or-user>/AuthDrift.git
git push -u origin main
```

Protect `main` on GitHub: **Repo → Settings → Branches → Add branch protection rule** → require PR reviews, disable direct pushes.

### B. Everyone — clone + create your local branch

```
git clone https://github.com/<org-or-user>/AuthDrift.git
cd AuthDrift
git checkout main
git pull origin main
git checkout -b balaa/assurance-detection
```

Swap the branch name for `tashi/keycloak-events`, `rohan/journey-ui`, or `varun/testing` as applicable. From this point every tool below opens *this same local folder, on this same branch*.

### 🔵 VS Code

```
code .
```

Open the cloned `AuthDrift` folder. Use the Source Control panel (or the integrated terminal) for the same `git add / commit / push` cycle below. The status bar bottom-left always shows your current branch — confirm it says `balaa/assurance-detection` before editing. Stage/commit either via terminal or the Source Control UI (+ icon to stage, checkmark to commit).

### 🟣 Claude Code

```
# Install once (Node.js 22+ required)
npm install -g @anthropic-ai/claude-code

# From inside your branch folder
cd AuthDrift
claude
```

First run opens a browser to authenticate (Claude.ai account or API key). Once inside, work conversationally on your branch's scope, e.g.:

```
Implement the assurance-assignment mapping in backend/assurance/
following the schema in docs/, then run the tests.
```

Claude Code can stage, commit, and push for you when asked, or you can drop to the same terminal commands manually. Run `/init` once to generate a `CLAUDE.md` project guide, and commit it.

### ⚫ Codex CLI

```
# Install once
npm install -g @openai/codex
```

```
# or: brew install codex

# From inside your branch folder
cd AuthDrift
codex
```

Sign in with your ChatGPT account when prompted. Give it a task scoped to your branch, e.g. "implement Keycloak event export and normalize into the common schema." Codex edits files and can run commands in a sandboxed working directory; review the diff it proposes before approving.

🟢 **Antigravity**

Download and install from `antigravity.google/download`, sign in with Google, then:

```
# From inside your branch folder
antigravity .
```

Or open the folder from Antigravity's start screen (it's a VS Code fork — importing your existing VS Code settings/extensions is offered on first launch). Confirm the active Git branch shown in the status bar, choose an autonomy level (Agent-assisted is the safer default for shared branches), and assign the agent your branch's scope of work. Review its generated Artifacts (plan, diffs, screenshots) before accepting changes.

### C. Daily working cycle (same in every tool's terminal)

```
git status
git add backend/assurance/assurance_model.py
git commit -m "Add initial passkey/OTP/password assurance mapping"
git push -u origin balaa/assurance-detection   # first push on this branch
git push                                        # subsequent pushes
```

### D. Keep your branch synced with main

```
git checkout main
git pull origin main
git checkout balaa/assurance-detection
git merge main
# resolve any conflicts, then:
git add .
git commit -m "Merge main into balaa/assurance-detection"
git push
```

### E. Opening a Pull Request

1. Push your branch: `git push -u origin <branch-name>`
2. GitHub → **Pull Requests → New Pull Request**
3. Base: `main` ← Compare: your branch
4. Describe what changed, which interface/schema it touches, how it was tested
5. Request review from the teammate sharing that interface (Balaa ↔ Tashi for the event schema, Balaa ↔ Rohan for journey/detection output)

### F. Merging (maintainer, after approval)

```
git checkout main
git pull origin main
git merge --no-ff tashi/keycloak-events -m "Merge: Keycloak setup + event normalizat
git push origin main
```

*(In practice, merge via the GitHub PR "Merge" button — it keeps the review record attached. The commands above are the manual equivalent.)*

### G. After merging — run the full test suite

```
git checkout main
git pull origin main
pytest tests/
```

### H. Cleanup

```
git branch -d balaa/assurance-detection
git push origin --delete balaa/assurance-detection
git log --oneline --graph --all
```

## 12. Recommended Repository Structure

```
AuthDrift/
├── backend/
│   ├── models/
│   ├── normalization/
│   ├── assurance/
│   ├── journey/
│   └── detection/
├── keycloak/
│   ├── configuration/
│   └── scenarios/
├── dashboard/
├── tests/
├── sample-data/
├── docs/
├── requirements.txt
└── README.md
```

> Do not create a large empty architecture just for appearance. Add directories as their implementation becomes necessary.

## 13. Development Sequence

| Phase | Work | Primary People |
|---|---|---|
| 0 | Freeze architecture, common event schema, branch responsibilities | All |

| | | |
|---|---|---|
| 1 | Keycloak setup and controlled authentication events | Tashi |
| 2 | Event extraction and normalization | Tashi + Balaa |
| 3 | Assurance model and assignment | Balaa |
| 4 | Journey reconstruction and timeline | Balaa + Rohan |
| 5 | Minimal assurance-drift detection | Balaa |
| 6 | Testing and regression suite | Varun + all |
| 7 | Integration into main and complete demonstration | All |

## 14. Initial Test Scenarios

| ID | Scenario | Expected Prototype Behaviour |
|---|---|---|
| TC01 | Known-device passkey success | Event collected, normalized, assigned high relative assurance, journey reconstructed; no drift |
| TC02 | Passkey → password fallback | Both events appear in one journey; assurance decreases; minimal drift detector flags it |
| TC03 | Passkey failure → password success | Failed strong method followed by weaker successful method is reconstructed and analysed |
| TC04 | Weaker authentication → new device | Journey contains the authentication event and device context if available |
| TC05 | Downgrade → new authenticator | Journey captures the downgrade and authenticator-registration context if available |

## 15. Final Prototype Demonstration

The strongest first demonstration is the **passkey-failure/fallback scenario**. Run the controlled authentication flow in Keycloak, show the raw event, show its normalized representation, show the assigned assurance, and then show the reconstructed journey.

```
KEYCLOAK
    ↓
```

```
Passkey attempt → FAILED
   ↓
Password fallback → SUCCESS
   ↓
Normalized events
   ↓
Assurance: Passkey = HIGH, Password = LOWER
   ↓
Authentication journey: HIGH → LOWER
   ↓
ASSURANCE DRIFT DETECTED
```

This is the minimum coherent story: the system does not merely log a password login; it understands that a stronger authentication attempt was followed by a weaker method within the same journey.

## 16. Definition of Authentication Assurance Drift

For this project, **authentication assurance drift** means a meaningful reduction in the relative authentication assurance during a user's authentication journey. A passkey-to-password transition is the basic example. Context such as device, location, method failure, or new authenticator registration can later increase the significance of that transition.

> This should not be confused with machine-learning "concept drift." AuthDrift's term is about a change in authentication assurance across the journey.

## 17. What We Should NOT Build First

- Machine-learning attack classification
- Production integration with real Microsoft/Google accounts
- Automatic account blocking or credential resets
- Enterprise-scale streaming infrastructure
- A complex scoring formula before the basic journey and drift logic work
- A sophisticated dashboard before the backend output is stable

## 18. Definition of Done for the First Prototype

- ☐ Keycloak produces controlled authentication events
- ☐ Events are converted into the agreed common schema
- ☐ Authentication methods receive relative assurance levels
- ☐ Events are grouped into a chronological user journey
- ☐ The prototype detects at least the basic strong-to-weaker transition
- ☐ The result can be reproduced using controlled test data
- ☐ All four branches have committed, reviewable work
- ☐ The integrated main branch can execute the complete demonstration

## 19. Extension After the Core Prototype

```
Drift Detection → Context Correlation → Risk Scoring → Explainable Alert
→ Dashboard → Second Provider/Mock Provider Format
```

These extensions move the project toward the remaining objectives in the synopsis. The prototype should first prove the core pipeline before additional complexity is introduced.

## 20. Source Basis

This plan is based on the submitted *AuthDrift: Authentication Assurance Drift Detection and Analysis* synopsis, especially its seven stated objectives, proposed methodology, controlled Keycloak scenarios, and expected outcomes.

The synopsis specifies that the system is intended as a lightweight, vendor-neutral monitoring layer that complements rather than replaces existing identity-provider policies, logs, and risk signals.